QML Lab • research project 10

Quantum Reinforcement Learning on CartPole

A variational quantum circuit as the policy of a policy-gradient agent.

Living document • last updated October 25, 2026

Quadrant	CQ (classical data, quantum processor)	**Track**	Hybrid
Difficulty	Hard	**Tools**	PennyLane, Gymnasium, PyTorch
Builds on	Modules 5 and 7 of the QML Lab	**Workspace**	`qml_projects/quantum-rl-cartpole/`

Summary. In reinforcement learning an agent learns from rewards rather than labels. This project replaces the policy network of a REINFORCE agent with a parameterised quantum circuit and trains it to balance the CartPole. It reproduces the design choices that make quantum policies work, input scaling and data re-uploading, and compares sample efficiency with a classical policy of similar size.

1 Background and key concepts

1.1 Policy gradients

An agent in state s_t takes action $a_t \sim \pi_\theta(a \mid s_t)$ and receives reward r_t. REINFORCE (Williams, 1992) maximises the expected return with the gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\big(G_t - b\big)\Big], \qquad G_t = \sum_{k\geq 0} \gamma^k r_{t+k}, \tag{1}$$

where b is a baseline that reduces variance (Sutton and Barto, 2018).

1.2 Quantum policies

Jerbi et al. (2021) define a softmax quantum policy

$$\pi_\theta(a \mid s) = \frac{e^{\beta \langle O_a \rangle_{s,\theta}}}{\sum_{a'} e^{\beta \langle O_{a'} \rangle_{s,\theta}}}, \tag{2}$$

where $\langle O_a \rangle_{s,\theta}$ is the expectation value of an observable assigned to action a after a circuit that encodes s with trainable scaling, $R_X(\lambda_i s_i)$, and re-uploads it in every layer; β is a trainable inverse temperature. They found input scaling and re-uploading essential for performance. Skolik et al. (2022) built the corresponding quantum deep Q-learning agent.

1.3 CartPole

The cart-pole balancing task (Barto et al., 1983), in its Gymnasium version `CartPole-v1` (Towers et al., 2024), has a 4-dimensional state (cart position and velocity, pole angle and angular velocity), two actions and a reward of 1 per time step, up to 500 steps. It counts as solved at an average return of 475 over 100 episodes.

2 Research questions

The project is open-ended: these questions are the starting point, not the finish line. Add new ones to the README and the journal as they come up.

RQ1 How many episodes does the quantum policy need to solve CartPole compared with a classical policy of similar size?

RQ2 Which matters more for the quantum policy: trainable input scaling or data re-uploading?

RQ3 How sensitive is learning to the number of layers?

3 Method

3.1 Data

Episodes generated by the agent in `CartPole-v1`. States are normalised with fixed bounds before encoding. Each update uses a batch of 10 episodes.

3.2 Models

4 qubits, L layers; each layer encodes the scaled state with $R_X(\lambda_i s_i)$, applies trainable R_Y/R_Z rotations and a ring of CZ gates. Two-qubit Z observables, one per action, and a trainable β form the softmax policy. Separate Adam learning rates for scaling parameters, rotation angles and β; discount $\gamma = 0.99$; return normalisation as baseline.

3.3 Baselines

A classical policy MLP $4 \to h \to 2$ with matched parameter count; a random policy; the published learning curves of Jerbi et al. (2021) as a reference.

3.4 Metrics

- Average return over the last 20 episodes as a function of episodes.
- Episodes needed to reach an average return of 475.
- Spread across 5 seeds; number of parameters; simulation time.

4 Planned experiments

Each experiment becomes one `[[experiments]]` entry in the workspace, so its runs, parameters and metrics are stored and compared in the portal.

5 Running it in the QML Lab

The workspace lives in `qml_projects/quantum-rl-cartpole/`. Scripts import three helpers from `qml_lab/plots.py`:

ID	Experiment	What varies	Main metrics
E1	Full quantum policy	$L = 5$, scaling + re-uploading	episodes to solve
E2	No input scaling	$\lambda_i = 1$ fixed	learning curve
E3	No re-uploading	encode once	learning curve
E4	Depth	$L \in \{2, 3, 5, 7\}$	episodes to solve
E5	Q-learning variant	DQN-style agent	sample efficiency

Table 1: Experiment plan. Results go into the journal and the results log (section 9).

- `params()` reads the parameters from the portal's run form (defaults when run by hand);
- `save()` stores figures with the run;
- `log_metrics()` records numbers that appear in the run table.

```
qml_projects/quantum-rl-cartpole/
  project.toml          metadata + experiments
  README.md             goal, open questions, ideas
  experiments/          runnable scripts
  docs/main.tex         this document
```

First experiment: This experiment is *planned*: write the script, add the block below to `project.toml`, then run `python manage.py sync_projects`.

```
[[experiments]]
slug = "reinforce"
title = "REINFORCE with a softmax quantum policy"
script = "experiments/e01_reinforce.py"
figure = "learning_curve.png"

[experiments.params]
n_layers = 5
episodes = 500
batch_episodes = 10
lr = 0.01
gamma = 0.99
seed = 0
```

```
# from the portal folder: pick up changes to project.toml
python manage.py sync_projects
cd qml_projects/quantum-rl-cartpole
# run by hand with the default parameters
PYTHONPATH=../../qml_lab python experiments/e01_reinforce.py
```

6 Quality bar

- A classical baseline with a similar number of trainable parameters, tuned with the same care (Bowles et al., 2024).
- At least five random seeds per configuration, reported as mean $\pm$ standard deviation.
- How results change with the number of qubits.
- At least one run under a realistic noise model.

- Report full learning curves with seed spread, not only the best run.

7 Open directions

- Other environments (Acrobot, FrozenLake).
- Policies estimated from finite shots, as on real hardware.
- Run the trained policy on hardware for inference only.

8 Risks and limitations

- RL results have high variance; five seeds is the minimum.
- Simulation is slow: every environment step needs a circuit evaluation.
- Hyperparameters (learning rates, β initialisation) strongly affect results.

9 Results log

Summarise findings here as they become stable; day-to-day notes belong in the portal journal.

Date	Run	Finding	Next step

References

Barto, A. G., Sutton, R. S. and Anderson, C. W. (1983) 'Neuronlike adaptive elements that can solve difficult learning control problems', *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5), pp. 834–846. Available at: https://doi.org/10.1109/TSMC.1983.6313077.

Bowles, J., Ahmed, S. and Schuld, M. (2024) 'Better than classical? The subtle art of benchmarking quantum machine learning models', Preprint, arXiv:2403.07059. Available at: https://arxiv.org/abs/2403.07059.

Jerbi, S., Gyurik, C., Marshall, S., Briegel, H. J. and Dunjko, V. (2021) 'Parametrized quantum policies for reinforcement learning', in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*.

Skolik, A., Jerbi, S. and Dunjko, V. (2022) 'Quantum agents in the Gym: a variational quantum algorithm for deep Q-learning', *Quantum*, 6, 720. Available at: https://doi.org/10.22331/q-2022-05-24-720.

Sutton, R. S. and Barto, A. G. (2018) *Reinforcement Learning: An Introduction*, 2nd edn. Cambridge, MA: MIT Press.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U. et al. (2024) 'Gymnasium: A standard interface for reinforcement learning environments', Preprint, arXiv:2407.17032. Available at: https://arxiv.org/abs/2407.17032.

Williams, R. J. (1992) 'Simple statistical gradient-following algorithms for connectionist reinforcement learning', *Machine Learning*, 8(3), pp. 229–256. Available at: https://doi.org/10.1007/BF00992696.